

PASSWORD! AT THE DISCO:

Uncovering Authentication Chaos in the Enterprise

Ryan Watson - watson0x90

Ancestry Red Team Lead



WHOAMI

Ryan Watson - watson0x90

Ancestry Red Team Lead

- Red Team lead focusing on enterprise security
- 10+ years in offensive security
- Physical Entry Specialist
- Advocate for proactive security assessments

WHAT WE'LL COVER

- **NTLM 101** - Understanding NTLM password hashing, and cracking techniques
- **The Problem** - Why traditional audits fall short
- **Password!AtTheDisco** - A better approach to password auditing
- **Architecture & Features** - BloodHound integration, HIBP correlation, risk scoring
- **Real-World Results** - Case studies and metrics
- **Implementation** - Getting started with the tool
- **Key Takeaways** - What you need to remember



NTLM 101

Understanding NTLM password hashing
and password cracking techniques

NTLM PASSWORD HASHING 101

- **Why does it exist?** - Windows domain authentication since NT 3.1 (1993)
- **What is NTLM?** - NT LAN Manager hash, MD4 hash of password (no salt!)
- **Where is it stored?** - Domain Controller's NTDS.dit database (SAM for local)
- **Why is it weak?** - No salt, fast to compute, billions of guesses per second with GPUs
- **What does "cracking" mean?** - Reversing hash → plaintext using brute force/wordlists

WHY DOES NTLM EXIST?

1993 ©

WINDOWS NT 3.1

- ▶ NTLM introduced for domain authentication
- ▶ Replaced earlier LM (LAN Manager) hash
- ▶ MD4 algorithm chosen for speed over security

2000 ©

WINDOWS 2000

- ▶ Kerberos becomes default protocol
- ▶ NTLM still stored in NTDS.dit for backward compatibility
- ▶ Microsoft recommends Kerberos migration

2025 ©

TODAY

- ▶ NTLM still required for legacy support
- ▶ Cannot be fully disabled in most environments
- ▶ Workgroup (non-domain) authentication
- ▶ Fallback when Kerberos fails
- ▶ Local account authentication (SAM database)

KEY TAKEAWAYS

- ▶ **Legacy Protocol** - 30+ years old, designed before modern threats
- ▶ **No Salt** - Identical passwords = identical hashes
- ▶ **Cannot Disable** - Required for backward compatibility
- ▶ **Still Stored** - Even when Kerberos is primary protocol
- ▶ **Attack Surface** - Every domain user is vulnerable

WHAT IS NTLM?

HASH STRUCTURE

Algorithm:

$\text{NTLM} = \text{MD4}(\text{UTF-16LE}(\text{password}))$

Steps:

1. Convert password to UTF-16LE
2. Apply MD4 hash function
3. Output 128-bit hash (32 hex chars)

Example:

Password: Fall2025!

$\text{MD4}(\text{UTF-16LE}(\text{Fall2025!}))$

Output: 6EF258AD585CAAA1675D7D88DA564909

KEY PROPERTIES

- ▶ **MD4 hash** of UTF-16LE encoded password
- ▶ **No salt** - major cryptographic weakness!
- ▶ **Same password = same hash** across all systems
- ▶ **32 hexadecimal characters** (128 bits)
- ▶ **Cryptographically broken** since 1996
- ▶ **Fast computation**
 - ▶ Fast on CPU
 - ▶ Extremely fast on GPU (billions/s+)

WHERE IS NTLM STORED?

DOMAIN CONTROLLERS

- **File:** C:\Windows\NTDS\ntds.dit
- **Contains:** All domain user hashes, computer account hashes, service account hashes, group memberships, trust relationships
- **Access Required:** SYSTEM privileges, Domain Admin rights, or DCSync attack

LOCAL SYSTEMS

- **Registry:** HKLM\SAM\SAM\Domains\Account\Users
- **Contains:** Local account hashes, administrator hash, cached domain credentials (optional)
- **Access Required:** Local Administrator, SYSTEM privileges, or offline attack

Extraction tools: Impacket secretsdump.py, Mimikatz, ntdsutil, Volume Shadow Copy

WHERE IS NTLM STORED? (CONT.)

ACTIVE MEMORY (LSASS PROCESS)

- **Location:** LSASS.exe process memory
- **Contains:** Recently authenticated user credentials, NTLM hashes of logged-on users, Kerberos tickets
- **Access Required:** SYSTEM or SeDebugPrivilege to read process memory, often requires local admin
- **Persistence:** Hashes remain in memory until user logs off or system reboots

BACKUP FILES & VOLUME SHADOW COPIES

- **Locations:** NTDS.dit backups, SAM registry hive exports, Volume Shadow Copy snapshots (VSS)
- **Contains:** Historical snapshots of domain/local credentials, potentially old/changed passwords
- **Access Required:** Backup operators, administrators, or filesystem-level access to VSS
- **Risk:** Backups often overlooked in security audits, can contain years-old credential data

Extraction tools: Mimikatz (sekurlsa::logonpasswords), ProcDump, Task Manager memory dumps, vssadmin

WHAT DOES "CRACKING" MEAN?

HASH → PLAINTEXT RECOVERY

Hashing algorithms are one-way, but attackers can retrieve the plaintext through systematic guessing:

- **Attack Strategies** - Bruteforce, or use simple wordlists to intelligent rule-based mutations and pattern matching.
- **Hardware Acceleration** - Why GPUs crack 100-1000x faster than CPUs
- **Offline Nature** - Once extracted, hashes can be attacked indefinitely without detection

Common tools: Hashcat, John the Ripper, oclHashcat, custom GPU scripts

BRUTEFORCE INFOGRAPH EXPLAINED...

Time it takes a hacker to brute force your password in 2025

Hardware: 12 x RTX 5090 | Password hash: bcrypt (10)

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	57 minutes	2 hours	4 hours
6	Instantly	46 minutes	2 days	6 days	2 weeks
7	Instantly	20 hours	4 months	1 year	2 years
8	Instantly	3 weeks	15 years	62 years	164 years
9	2 hours	2 years	791 years	3k years	11k years
10	1 day	40 years	41k years	238k years	803k years
11	1 weeks	1k years	2m years	14m years	56m years
12	3 months	27k years	111m years	917m years	3bn years
13	3 years	705k years	5bn years	56bn years	275bn years
14	28 years	18m years	300bn years	3tn years	19tn years
15	284 years	477m years	15tn years	218tn years	1qd years
16	2k years	12bn years	812tn years	13qd years	94qd years
17	28k years	322bn years	42qd years	840qd years	6qn years
18	284k years	8tn years	2qn years	52qn years	463qn years



Hive Systems

Read more and download at
hivesystems.com/password



BRUTEFORCE INFOGRAPH EXPLAINED...

Time it takes a hacker to brute force your password in 2025

Hardware: 12 x RTX 5090 | Password hash: bcrypt (10)

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	57 minutes	2 hours	4 hours
6	Instantly	46 minutes	2 days	6 days	2 weeks
7	Instantly	20 hours	4 months	1 year	2 years
8	Instantly	3 weeks	15 years	62 years	164 years
9	2 hours	2 years	791 years	3k years	11k years
10	1 day	40 years	41k years	238k years	803k years
11	1 weeks	1k years	2m years	14m years	56m years
12	3 months	27k years	111m years	917m years	3bn years
13	3 years	705k years	5bn years	56bn years	275bn years
14	28 years	18m years	300bn years	3tn years	19tn years
15	284 years	477m years	15tn years	218tn years	1qd years
16	2k years	12bn years	812tn years	13qd years	94qd years
17	28k years	322bn years	42qd years	840qd years	6qn years
18	284k years	8tn years	2qn years	52qn years	463qn years



Hive Systems

Read more and download at
hivesystems.com/password

PASSWORD CRACKING WITH BCRYPT

What is this?

This chart shows cracking times for **bcrypt** - a **modern, secure** password hashing algorithm.

Key Example:

12-character password (mixed case + symbols)

Time to crack: **3 billion years**

Why Are We Showing You This?

Because NTLM has NONE of these protections.

This chart is **completely irrelevant** to NTLM. The same passwords that take billions of years in bcrypt... can be cracked in **hours or days** with NTLM.

Let's understand WHY bcrypt takes so long - and why NTLM doesn't.



ALGORITHM COMPLEXITY: MD4 VS BCRYPT

MD4 ALGORITHM

Round 1 - Selection:

$$F(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z)$$

Round 2 - Majority:

$$G(X, Y, Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z)$$

Round 3 - Parity:

$$H(X, Y, Z) = X \oplus Y \oplus Z$$

Operation:

$$a = (a + \phi(b, c, d) + X[k] + K_i) \lll s$$

Takeaways:

- ▶ 3 rounds × 16 operations = **48 total**
- ▶ No salt (same constants every time)
- ▶ Designed for speed in 1990

Key Point: MD4's simplicity and lack of computational cost make it extremely fast to compute, but also trivially breakable with modern hardware.

BCRYPT ALGORITHM

Cost-Based Iterations:

$$\text{iterations} = 2^{\text{cost}}$$

Example (cost = 10):

$$2^{10} = 1,024 \text{ iterations}$$

Eksblowfish Setup:

$$P, S \leftarrow \text{EksBlowfishSetup}(\text{pwd}, \text{salt}, \text{cost})$$

64× Encryption:

$$\text{Encrypt}(\text{"OrpheanBeholderScryDoubt"}) \times 64$$

Takeaways:

- ▶ 1,024+ iterations with salt
- ▶ P-array (18 × 32-bit) + S-boxes (4 × 256 × 32-bit)
- ▶ Computationally expensive by design

Key Point: bcrypt's exponential cost factor and use of salt make it highly resistant to brute-force attacks, significantly slowing down attackers even with powerful hardware.



HASHCAT ATTACK STRATEGY HIERARCHY

FROM SMART TO EXHAUSTIVE

- **Dictionary Attack** - Test known passwords from wordlists (most efficient starting point)
- **Rule-Based Attack** - Apply transformations to wordlist entries (password → P@ssw0rd2024!)
- **Hybrid Attack** - Combine wordlists with patterns (wordlist + ?d?d?d?d)
- **Mask Attack** - Target specific patterns with intelligence (?u?!?!?!?d?d)
- **Brute Force** - Try every combination (impractical for 8+ characters)

Strategy: Always start smart, progress to exhaustive only when necessary



DICTIONARY ATTACKS & WORDLISTS

POPULAR WORDLISTS

RockyOU.txt

- ▶ 14.3 million passwords
- ▶ From 2009 data breach
- ▶ Most common starting point

Weakpass Collections

- ▶ Multiple sizes (1M to 10GB+)
- ▶ 57.5% crack rate on enterprise datasets
- ▶ Regularly updated from breaches

CrackStation

- ▶ 15GB wordlist (1.5 billion entries)
- ▶ Aggregated from multiple sources
- ▶ Used for exhaustive dictionary testing

ATTACK EFFECTIVENESS

Success Rates:

- ▶ Varies widely (10-50%) based on password policies
- ▶ Most common: password123, letmein, qwerty
- ▶ Corporate: CompanyName123, Season+Year

Why It Works:

- ▶ Humans choose memorable passwords
- ▶ Password reuse across services
- ▶ Leaked passwords become public knowledge

Speed:

- ▶ GPU-limited, not wordlist-limited
- ▶ Billions of candidates per second
- ▶ 14M RockyOU tested in seconds



RULE-BASED ATTACKS

COMMON RULE OPERATIONS

Case Transformations:

- ▶ password → Password (capitalize)
- ▶ password → PASSWORD (uppercase)
- ▶ password → pAssWorD (toggle case)

Leetspeak Substitution:

- ▶ password → p@ssw0rd (a → @, o → 0)
- ▶ password → pa55word (s → 5)
- ▶ elite → 3l1t3 (e → 3, i → 1)

Append/Prepend:

- ▶ password → password123 (append digits)
- ▶ password → password! (append special)
- ▶ password → 2024password (prepend year)

POPULAR RULE SETS

best64.rule

- ▶ 64 optimized rules (ships with hashcat)
- ▶ Balanced efficiency/coverage
- ▶ Recommended starting point

OneRuleToRuleThemAll

- ▶ 52,000 rules
- ▶ 68% crack rate on leaked datasets
- ▶ Based on real password patterns

dive.rule

- ▶ Aggressive transformations
- ▶ Combines multiple operations
- ▶ Used after initial attempts fail

Command:

```
hashcat -a 0 -m 1000 hashes.txt rockyou.txt -r best64.rule
```



MASK ATTACKS & PATTERNS

CHARACTER SETS (BUILT-IN)

- ▶ **?l** = lowercase (a-z, 26 chars)
- ▶ **?u** = uppercase (A-Z, 26 chars)
- ▶ **?d** = digits (0-9, 10 chars)
- ▶ **?s** = special chars (!@#\$, 32 chars)
- ▶ **?a** = all printable ASCII (95 chars)
- ▶ **?b** = all bytes (0x00-0xFF, 256 chars)

CUSTOM CHARACTER SETS

- ▶ **-1** = define custom set 1
- ▶ **-2** = define custom set 2
- ▶ Example: **-1 ?l?d** = lowercase + digits
- ▶ Usage: **?1?1?1?1** = 4 chars from set 1

Command:

```
hashcat -a 3 -m 1000 hashes.txt ?u?l?l?l?l?d?d
```

PATTERN EXAMPLES & KEYSPEC

Example: ?u?l?l?l?l?d?d

- ▶ Pattern: Capital + 4 lowercase + 2 digits
- ▶ Keyspace: $26 \times 26^4 \times 10^2 = 45$ billion
- ▶ Time (RTX 4090): ~3 minutes
- ▶ Matches: Password12, Company99, Welcome23

Policy Targeting:

- ▶ Policy: "8 chars, 1 upper, 1 number"
- ▶ Target: ?u?l?l?l?l?l?d?d
- ▶ Keyspace: $26 \times 26^5 \times 10^2 = 1.2$ trillion
- ▶ Time (RTX 4090): ~1.3 hours

vs Pure Brute Force:

- ▶ Pure brute (7 chars): $95^7 = 70$ trillion
- ▶ Mask attack (7 chars): 45 billion
- ▶ **1,555× reduction in keyspace!**

PERFORMANCE & OPTIMIZATION

GPU VS CPU PERFORMANCE

NTLM Hash Rate (GH/s):

- ▶ Intel i9-14900K (CPU): ~2 GH/s
- ▶ NVIDIA RTX 3090: ~122 GH/s
- ▶ NVIDIA RTX 4090: ~250 GH/s
- ▶ **GPU speedup: 100-125× faster**

MULTI-GPU SCALING

- ▶ Linear scaling: 4 GPUs = 4× speed
- ▶ 4× RTX 4090: ~1 TH/s (1 trillion/s)
- ▶ 8× RTX 4090: ~2 TH/s
- ▶ Command: `-d 1,2,3,4` (use GPUs 1-4)

Why GPUs?

- ▶ Parallel architecture (thousands of cores)
- ▶ Optimized for repetitive calculations
- ▶ MD4/NTLM are GPU-friendly algorithms

OPTIMIZATION FLAGS

-O (Optimized Kernels)

- ▶ Enables optimized GPU kernels
- ▶ ~2× speed improvement
- ▶ Limitation: Max 32 characters
- ▶ Recommended for NTLM hashes

-w (Workload Profile)

- ▶ `-w 1`: Low (desktop usable)
- ▶ `-w 2`: Default (balanced)
- ▶ `-w 3`: High (slow desktop)
- ▶ `-w 4`: Nightmare (max performance)

Session Management:

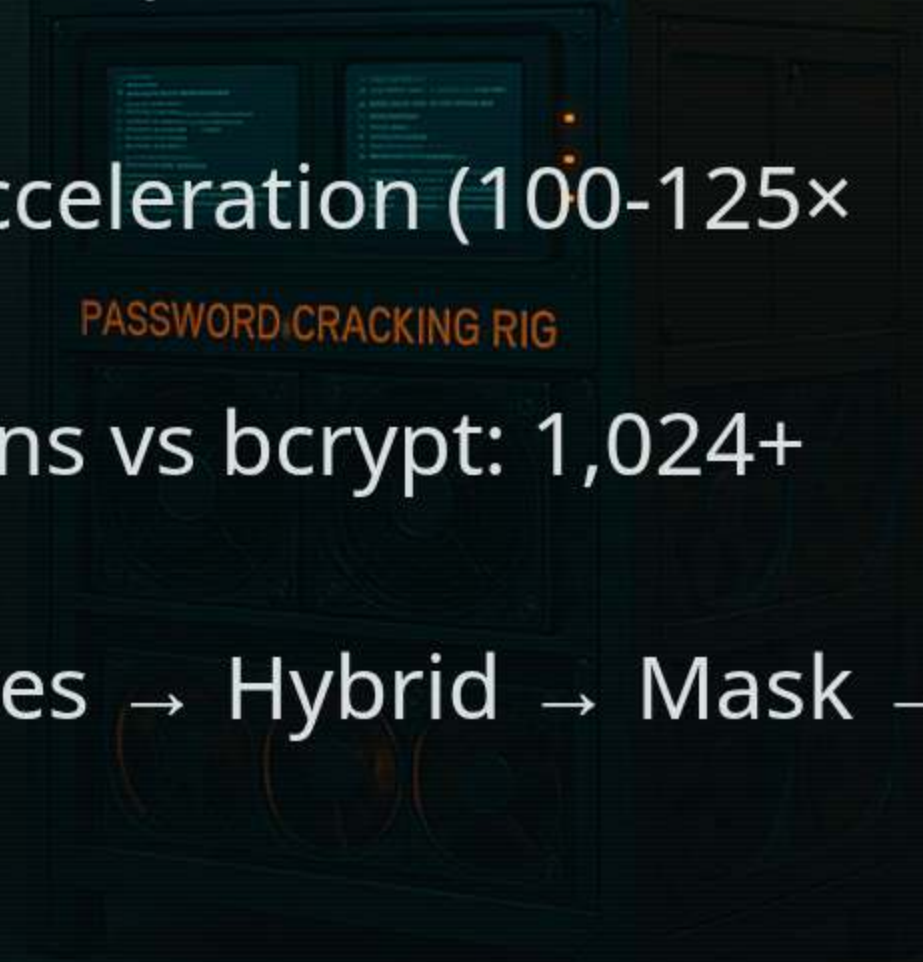
- ▶ `--session name`: Save progress
- ▶ `--restore`: Resume session
- ▶ Auto-saves every 10 seconds

Example:

```
hashcat -a 0 -m 1000 hashes.txt rockyou.txt -O -w 4 --session mycrack
```

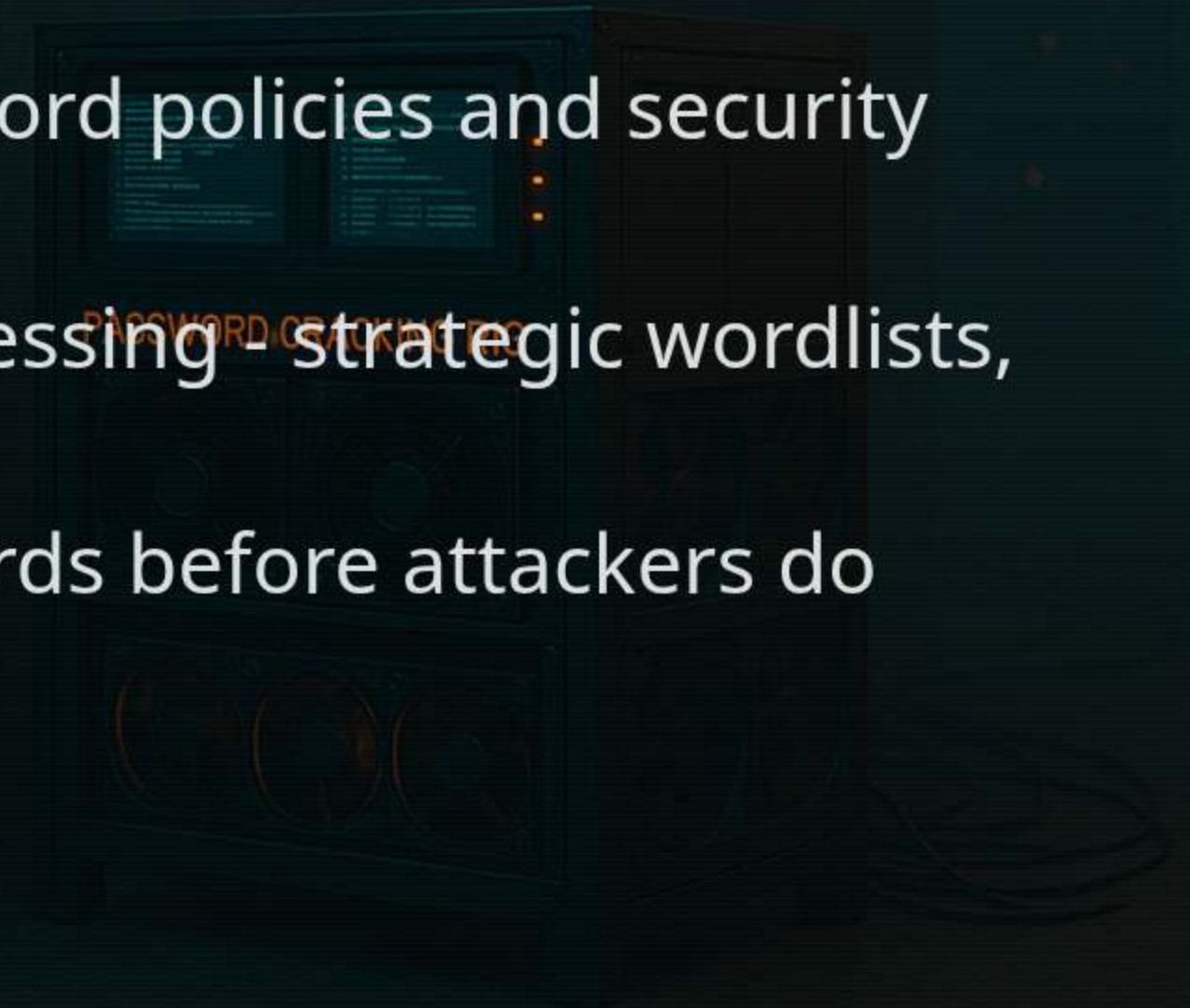

NTLM 101: WHAT WE COVERED

- **What is NTLM?** - MD4 hash of password, no salt, 32 hex characters, cryptographically broken since 1996
- **Why It Exists** - Legacy protocol from 1993, required for backward compatibility, can't be disabled
- **Where It's Stored** - Domain Controllers (NTDS.dit), Local SAM, Active Memory (LSASS), Backups/VSS
- **How Cracking Works** - Offline attacks, GPU acceleration (100-125× faster), billions of hashes per second
- **Algorithm Weakness** - MD4: only 48 operations vs bcrypt: 1,024+ iterations with salt
- **Hashcat Attack Strategies** - Dictionary → Rules → Hybrid → Mask → Brute Force (smart to exhaustive)



WHY THIS MATTERS

- **NTLM is Everywhere** - Every Windows domain, can't be disabled, billions of hashes exist
- **It's Extremely Weak** - No salt + fast algorithm = ideal for cracking
- **GPU Changes Everything** - Consumer hardware can test 250 billion hashes/second
- **Success Varies** - 10-50% depending on password policies and security maturity
- **Attackers Use Intelligence** - Not random guessing - strategic wordlists, rules, and patterns
- **This Is Why We Audit** - Identify weak passwords before attackers do





ACTIVE DIRECTORY PASSWORD POLICIES

Understanding the Foundation and Its Flaws

DEFAULT DOMAIN PASSWORD POLICY

DEFAULT SETTINGS

- ▶ **Minimum Length:** 7 characters
- ▶ **Complexity:** Enabled
- ▶ **Maximum Age:** 42 days
- ▶ **Password History:** 24 passwords
- ▶ **Account Lockout:** DISABLED
 - ▶ Threshold = 0

Critical Flaw: 7-character passwords can be cracked in 7 seconds with modern GPUs

WHAT "COMPLEXITY" ACTUALLY MEANS

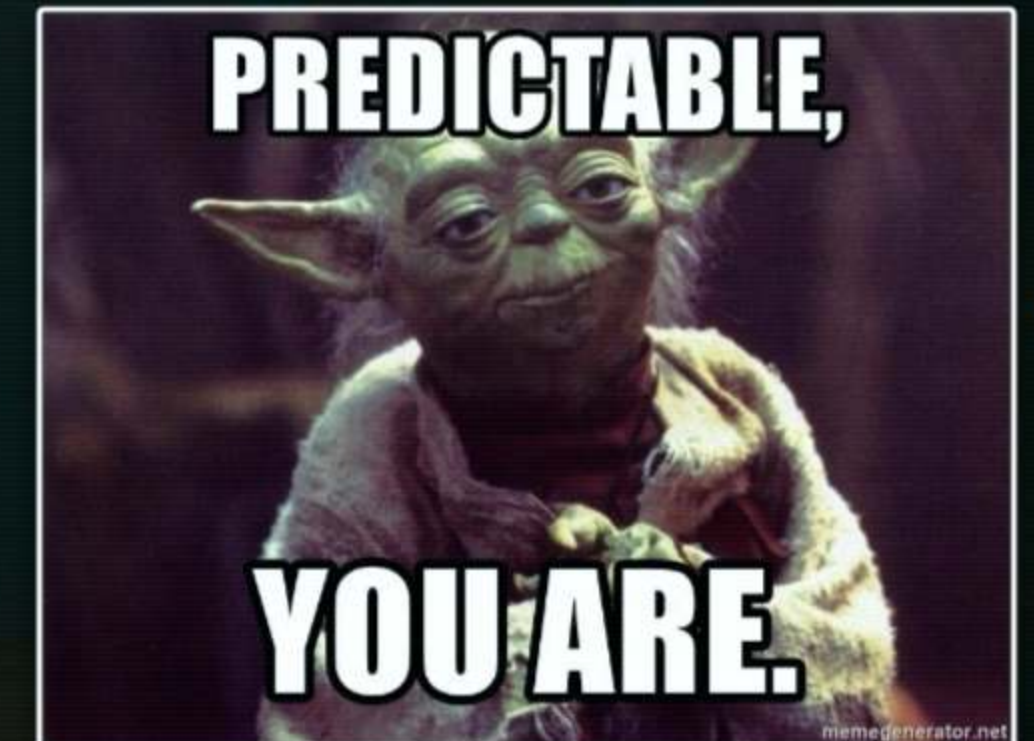
- ▶ Must **NOT** contain username or full name
- ▶ Must contain 3 of 4 categories:
 - ▶ Uppercase (A-Z)
 - ▶ Lowercase (a-z)
 - ▶ Numbers (0-9)
 - ▶ Symbols (!@#\$%)
- ▶ ✗ NO dictionary checking
- ▶ ✗ NO breach database checking
- ▶ ✗ NO pattern detection

THE COMPLEXITY PARADOX

82% of users create predictable patterns when complexity is mandated¹

- **NIST 2024 Reversal:** Eliminated requirements for special characters and mixed case²
 - Quote: *"Complexity rules reduce key space and cause users to act in predictable ways"*
- **Predictable User Behavior:**
 - Capital letter in first position (90%+ of users)
 - Symbol in last position (80%+ of users)
 - Number in last 2 positions (85%+ of users)
- **Real-World Example:** "Password123!" appears in millions of breaches³
 - Meets all complexity requirements ✓
 - First password attackers try ✗

The Paradox: Complexity requirements designed to increase security actually decrease it by creating predictable patterns attackers can exploit.⁴



COMMON PASSWORD PATTERNS

TOP PATTERNS FOUND IN ENTERPRISE

▶ Season + Year

- ▶ Summer2024!, Fall2024!, Winter2025!
- ▶ Rotates predictably with calendar

▶ Company + Number

- ▶ CompanyName123!, Microsoft2024!
- ▶ First guess for attackers

▶ Welcome Variations

- ▶ Welc0me123!, Welc0me2024!
- ▶ Users never changed default

▶ Keyboard Walks

- ▶ Qwerty123! - found 1M+ times in 800M breaches
- ▶ 1qaz2wsx!, Asdfgh123!

WHY THESE PATTERNS EMERGE

▶ Forced Password Rotation

- ▶ Users increment: Password1! → Password2!
- ▶ Research: 41% of new passwords guessable from previous

▶ Complexity Requirements

- ▶ Capital + number + symbol added to end
- ▶ Minimal effort to satisfy policy

▶ Cognitive Load Minimization

- ▶ Familiar words + policy requirements
- ▶ Easy to remember = easy to guess

▶ Attacker Intelligence

- ▶ Hashcat rules apply these exact transformations
- ▶ Your password policy becomes their attack roadmap



REAL-WORLD ATTACK: PREDICTABLE PASSWORD



INITIAL COMPROMISE

Password guessing against ADFS portal, trying only 3 different passwords. The password '**Fall2016!**' was successful for an IT Admin. Predictable seasonal pattern with year and special character.

CODE EXECUTION & LATERAL MOVEMENT

Code Execution via Outlook Ruler attack on the IT Admin's workstation. Found a server where the IT Admin had access and where a Domain Admin was logged in.

DOMAIN ADMIN ACCESS

Domain Admin privileges achieved. Full network compromise. Total time from initial compromise: **25 min...**



PASSWORD POLICY LIMITATIONS

BEFORE WINDOWS SERVER 2008 (15 YEARS!)

- **ONE password policy per domain**
 - Domain Admins = same requirements as interns
 - Forced "lowest common denominator" approach
 - Only solution: Create separate domains (massive overhead)

EVEN WITH MODERN POLICIES (FGPP, MICROSOFT ENTRA ID)

- **On-Premises Active Directory:**
 - ✗ **Cannot check against breach databases** - Zero native protection
 - ✗ **Cannot block dictionary words** - "Password123!" meets requirements
 - ✗ **Cannot detect context** - Company names, usernames pass policy
- **Microsoft Entra ID (Cloud):**
 - ⚠ **Limited breach protection**
 - Global list: ~500 passwords (Microsoft's telemetry)¹
 - Does NOT check comprehensive databases (800M+ passwords)²
 - 44 million Entra ID accounts still compromised³
 - ⚠ **Limited dictionary blocking**
 - Custom banned list: 1,000 terms max (requires P1/P2)⁴
 - Insufficient for comprehensive protection
 - ✗ **Cannot detect semantic context**
 - Basic username/tenant checking only



EXPLOITING WEAK POLICIES

PASSWORD SPRAYING

Accounts for a **THIRD** of account compromises in organizations¹

- Try 1-3 common passwords against **ALL** accounts
- Stays below lockout threshold
- Weak policies create the attack surface

WEAK PASSWORD REALITY

- Default AD policy: **7 characters** minimum, **NO account lockout**³
- **75%** of organizations: accounts using top 1,000 passwords⁴
- **15-20%** of corporate accounts use compromised passwords⁴
- **81%** of breaches involve weak/stolen credentials⁵

THE ATTACK SURFACE GAP

- **90%** of attacks target AD environments²
- Weak defaults + predictable patterns = **attacker roadmap**
- MFA reduces risk by **99.9%** but only **38%** of SMBs adopt it⁶
- **The gap between defaults and best practices is where attackers thrive**



FINE-GRAINED PASSWORD POLICIES (FGPP)

WHAT FGPP ENABLES

- ▶ **Multiple password policies** per domain
- ▶ **Different requirements** for different groups
 - ▶ Example: Admins 20+ chars, users 12+ chars
- ▶ Applied to **users and global security groups**
- ▶ **Precedence rules** for policy conflicts
- ▶ **Better than one policy** - but not a complete solution

WHAT FGPP STILL CAN'T FIX

- ▶ Cannot check **breach databases** (800M+ compromised passwords)
- ▶ Cannot block **dictionary words**
- ▶ Cannot apply to **OUs** (only users/groups)
- ▶ No **fuzzy matching** for banned passwords
- ▶ Doesn't prevent **predictable patterns**

Bottom Line: FGPP allows differentiation, but **users can still set weak, breached passwords** that meet all requirements.

THE PROBLEM

Why Traditional Password Audits Fall Short



TRADITIONAL PASSWORD AUDITING

THE STANDARD WORKFLOW

- **Extract Hashes** - Pull NTDS.dit from Domain Controller, extract password hashes
- **Run Hashcat** - Crack passwords using wordlists and rules (hours to days)
- **Generate Statistics** - Use tools like pipal to analyze patterns and trends
- **Manual Interpretation** - Security team reviews data, determines what's risky
- **Create Reports** - Spreadsheets, documents, lists of weak passwords
- **Remediation Tracking** - Manual follow-up, tickets, email campaigns



TOOLS LIKE PIPAL: HELPFUL BUT LIMITED

WHAT PIPAL DOES WELL

- ▶ Statistical analysis of password dumps
- ▶ Top passwords and character sets
- ▶ Password length distribution
- ▶ Pattern detection (years, months, days)
- ▶ Hashcat mask suggestions
- ▶ Quick insights into common patterns
- ▶ Base word analysis

Example Output:

```
Top 10 passwords
Password123 = 47 (2.35%)
Summer2024! = 31 (1.55%)
Welcome1    = 28 (1.40%)
```

WHAT'S MISSING

- ▶ **No Risk Context** - Is "Password123" worse for a DA or regular user?
- ▶ **No Privilege Awareness** - Can't identify high-value targets
- ▶ **No Lateral Movement Analysis** - Shared passwords across accounts?
- ▶ **No BloodHound Integration** - Which accounts have DA paths?
- ▶ **Manual Correlation Required** - You connect all the dots yourself
- ▶ **No Prioritization** - All weak passwords treated equally

Creator's own words:

"The real work is done by you in interpreting the results"

PIPAL OUTPUT EXAMPLE

Pipal v2.0 - Password Analysis Tool

Total entries: 2,000

Total unique entries: 1,847

Top 10 passwords

Password123 = 47 (2.35%)

Summer2024! = 31 (1.55%)

Welcome1 = 28 (1.40%)

Company2024 = 24 (1.20%)

P@ssw0rd = 19 (0.95%)

Winter2024 = 18 (0.90%)

Admin123 = 15 (0.75%)

Spring2025 = 14 (0.70%)

Welcome123! = 12 (0.60%)

Password1 = 11 (0.55%)

Question: Which of these 47 "Password123" users has a path to Domain Admin?

Pipal's answer: ㄟ(ˉ▽ˉ)ㄟ

THREE CRITICAL GAPS

1. CONTEXT COLLAPSE

Statistics without risk context aren't actionable

"Password123" appears 47 times... but is that actually a crisis?

2. THE PRIVILEGE PROBLEM

Weak password on DA account $\neq$ weak password on standard user

Risk is determined by **who has the password**, not just the password itself

3. THE SCALE PROBLEM

10,000 weak passwords? Which 10 do you fix first?

Without prioritization, you're either fixing everything (impossible) or nothing (dangerous)

THE MANUAL BURDEN

Scenario: You've completed a domain audit

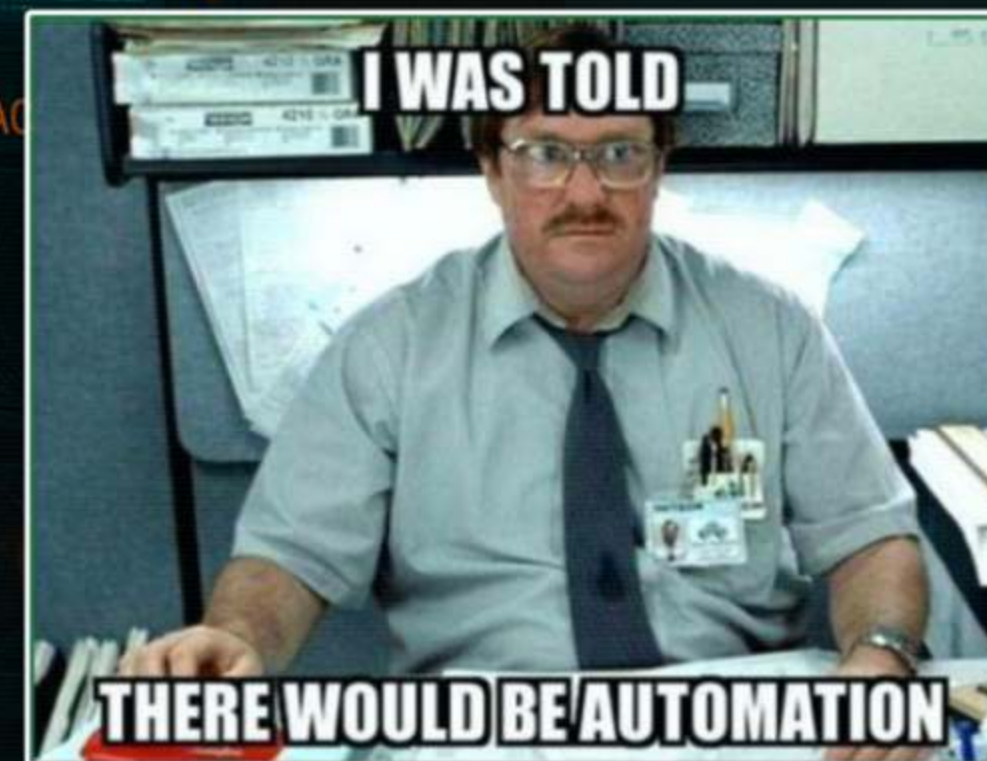
- 5,000 passwords cracked from 15,000 hashes
- Pipal shows "Summer2024!" used by 47 accounts

Questions you must answer manually:

- Which of these 47 users have local admin rights on critical servers?
- Do any have paths to Domain Admin in BloodHound?
- Are any service accounts with GenericAll permissions?
- How many domains in the forest share this password?
- Which accounts do I fix **TODAY** vs next month vs next quarter?

Reality: Hours of manual BloodHound queries, cross-referencing spreadsheets, and tribal knowledge

Outcome: Audit that took days to crack now takes multiple days to analyze - most teams simply don't have the time



PASSWORD! AT THE DISCO

Automated Risk-Based Password Auditing



PASSWORD! AT THE DISCO

THE MISSION

Bridge the gap between cracked passwords and actionable priorities

THREE PILLARS

- **Integration**
 - Hashcat output
 - Have I Been Pwned (1.3B breach database)
 - BloodHound privilege data
- **Automation**
 - Automatic BloodHound queries for every cracked account
 - No manual cross-referencing or spreadsheets
- **Intelligence**
 - CVSS-style three-component risk scoring
 - Clear guidance on what to fix first



CORE FEATURES

Lets make this actionable!



BLOODHOUND INTEGRATION

BLOODHOUND OVERVIEW

BloodHound is a powerful Active Directory analysis tool that maps out relationships and permissions in AD environments.

- Used by red teams and blue teams alike to understand AD security
- Visualizes paths to high-value targets like Domain Admins
- Reveals effective permissions, not just explicit group memberships

Github Repos:

- <https://github.com/SpecterOps/BloodHound>
- <https://github.com/SpecterOps/SharpHound>
- <https://github.com/SpecterOps/AzureHound>

FOR EVERY CRACKED ACCOUNT, AUTOMATICALLY QUERY:

- **Shortest Paths to Domain Admin** - How many hops to DA?
- **Outbound Object Control** - How many objects can this account control?
- **Group Memberships** - Including nested and recursive groups
- **Session Privileges** - Where does this account have active sessions?



BLOODHOUND INTEGRATION

BLOODHOUND OVERVIEW

BloodHound is a powerful Active Directory analysis tool that maps out relationships and permissions in AD environments.

- Used by red teams and blue teams alike to understand AD security
- Visualizes paths to high-value targets like Domain Admins
- Reveals effective permissions, not just explicit group memberships

Github Repos:

- <https://github.com/SpecterOps/BloodHound>
- <https://github.com/SpecterOps/SharpHound>
- <https://github.com/SpecterOps/AzureHound>

FOR EVERY CRACKED ACCOUNT, AUTOMATICALLY QUERY:

- **Shortest Paths to Domain Admin** - How many hops to DA?
- **Outbound Object Control** - How many objects can this account control?
- **Group Memberships** - Including nested and recursive groups
- **Session Privileges** - Where does this account have active sessions?

Note: You will need to run BloodHound ahead of time to populate the database.



HAVE I BEEN PWNEED: BREACH DATABASE



WHAT IS HAVE I BEEN PWNEED?

- **Free Service** - Check if your data was compromised in breaches
- **Pwned Passwords** - Downloadable database of 1B+ compromised passwords
- **Privacy-First** - k-Anonymity API protects what you're checking
- **Trusted** - Law enforcement partnerships (FBI, UK NCA)



WHY HIBP IS LEGITIMATE

TRUSTED LEADERSHIP

- **Troy Hunt** - Microsoft Regional Director & MVP
- Respected security researcher & author
- Transparent about data sources & methodology

GOVERNMENT PARTNERSHIPS

- **FBI** - Contributed 320M+ passwords
- **UK NCA** - Contributed 265M+ passwords
- Law enforcement wouldn't partner with illegal sites

INDUSTRY ADOPTION

- **Microsoft** - Integrated into Azure AD
- **1Password, Cloudflare, GitHub**
- **NIST SP 800-63B** recommended



kbslacker

WHY HIBP IS LEGITIMATE

TRUSTED LEADERSHIP

- **Troy Hunt** - Microsoft Regional Director & MVP
- Respected security researcher & author
- Transparent about data sources & methodology

GOVERNMENT PARTNERSHIPS

- **FBI** - Contributed 320M+ passwords
- **UK NCA** - Contributed 265M+ passwords
- Law enforcement wouldn't partner with illegal sites

INDUSTRY ADOPTION

- **Microsoft** - Integrated into Azure AD
- **1Password, Cloudflare, GitHub**
- **NIST SP 800-63B** recommended

PRIVACY-FIRST DESIGN

- **k-Anonymity API** - No plaintext exposure
- Downloadable database (offline checking)
- No tracking or user data collection



WHY IT'S NOT DARKWEB

SOURCE TRANSPARENCY

- **Publicly disclosed breaches ONLY**
- Every source is documented & verified
- No "underground" or illegal data purchases

LEGAL & ETHICAL

- Data from **public domain** breaches
- No hacking or unauthorized access
- Law enforcement contributions = legal validation

PURPOSE & INTENT

- **Defensive security** tool
- Helps users protect themselves
- Enables proactive password auditing

COMPLIANCE TOOL

- **NIST guidelines** explicitly recommend it
- Used for regulatory compliance (PCI-DSS, etc.)
- Endorsed by security community worldwide

Key Distinction: Darkweb sites *sell* stolen credentials for criminal purposes. HIBP *freely provides* breach data to help organizations and individuals **defend** against attacks.



PWNED PASSWORDS: NTLM DATABASE

WHAT IS PWNED PASSWORDS?

- **1+ billion unique passwords** from verified breaches
- Downloadable database for **offline checking**
- Available in two formats: **SHA-1** and **NTLM**
- Updated regularly as new breaches are disclosed

WHY NTLM VERSION?

- Windows domains store passwords as **NTLM hashes**
- Direct comparison: No need to crack hashes first
- Enables **instant risk assessment** of extracted hashes
- Perfect for Active Directory audits

PASSWORD CRACKING RIG

SECURE
YOUR
PASSWORDS!



HIBP INTEGRATION ARCHITECTURE

1.3 BILLION BREACHED PASSWORD DATABASE

- **Have I Been Pwned (HIBP)** - Troy Hunt's breach database
- 42GB local file of NTLM password hashes
- Downloaded from <https://haveibeenpwned.com/Passwords>
- **NOT an API integration** - fully offline after download

THREE-TIER LOOKUP SYSTEM

TIER 1: IN-MEMORY CACHE

- 1 million most common breached hashes
- **<1ms** lookup time
- ~95% hit rate for typical enterprise audits

TIER 2: BINARY SEARCH INDEX

- Built during first run (5-10 minutes one-time)
- **10-50ms** lookup time
- Handles remaining ~4% of passwords

TIER 3: SEQUENTIAL FILE SCAN

- Fallback for edge cases
- **~1 second** lookup time
- Rarely needed (<1% of lookups)

Performance: 10,000 passwords checked in ~30 seconds (after first-run indexing)

Privacy: Zero network calls, zero telemetry, 100% offline



ADDITIONAL ANALYSIS METHODOLOGY

AUTOMATED PASSWORD CHECKS

Dictionary Word Detection

- ▶ **~479,000 English words** from multiple sources
- ▶ Case-insensitive matching
- ▶ Detects: "Dragon123", "Sunshine2024!", "Football99"
- ▶ Partial matching: "DragonSlayer" contains "Dragon"

Common Password Lists

- ▶ **Top 10,000** most common passwords globally
- ▶ SecLists, RockYou, Daniel Miessler collections
- ▶ Seasonal patterns: "Summer2024", "Winter2023"
- ▶ Default passwords: "admin", "changeme", "welcome"

Keyboard Patterns

- ▶ **QWERTY sequences**: "qwertyuiop", "asdfghjkl"

ORGANIZATION-SPECIFIC CHECKS

Forbidden Words (Customizable)

- ▶ **Company name**: "Acme", "AcmeCorp"
- ▶ **Product names**: "Widget", "Gadget"
- ▶ **Locations**: "Boston", "Headquarters"
- ▶ **Departments**: "Finance", "Engineering"

Password Similarity Detection

- ▶ **Levenshtein distance** algorithm
- ▶ Threshold: >80% similarity flagged
- ▶ Detects variations: "Password123" → "Password124"
- ▶ Identifies user password rotation patterns

Complexity & Length Analysis

- ▶ **16 complexity categories**: Best → VeryLow



A dimly lit room, possibly a classroom or meeting room, with a wooden table and several chairs. A single light bulb hangs from the ceiling, casting a warm glow. In the background, there is a large projector screen and a whiteboard with some handwritten notes. The overall atmosphere is quiet and focused.

SEE IT IN ACTION

Demo & Examples



SEE IT IN ACTION

LIVE DEMONSTRATION

Let's explore the actual project and reports

WHAT WE'LL COVER

- **GitHub Repository Walkthrough**
 - Project structure and architecture
 - Documentation and usage guides
 - Installation and configuration
- **Interactive HTML Dashboard**
 - Security Posture Score (0-100)
 - FlexSearch-powered account search
 - Risk distribution visualizations
 - Dark mode and filtering features
- **Excel Actionable Reports**
 - DA Pathways sheet (START HERE)
 - Top 100 Risks, HIBP Breached accounts
 - Recommended Actions column
 - 8+ prioritized sheets for different audiences

Repository: github.com/passwordathedisco/PasswordAtTheDisco



BEST PRACTICES

AUDIT FREQUENCY

- **Minimum:** Quarterly audits
- **High-security:** Monthly audits
- **Post-incident:** Immediate re-audit

POLICY CONFIGURATION

- Align password_policy.json with your **actual** AD policy (not aspirational)
- Update when policies change (track in version control)
- Test policy changes in reports before enforcing in AD

PRIORITIZATION STRATEGY

- **Critical:** Fix within 24-48 hours (executives approve overtime if needed)
- **High:** Fix within 1 week (assign owners, track progress)
- **Medium:** Fix within 1 month (batch into sprint work)
- **Low:** Address in next audit cycle (user education campaigns)

COMMUNICATION & TRACKING

- Use **HTML reports** for executives (visual, charts, easy to understand)
- Use **CSV exports** for ticketing systems and remediation tracking
- Include actionable report excerpts in security briefings
- Track metrics over time: crack rate, risk scores, critical account trends

WHAT THE AUDIT TELLS YOU

COMMON PASSWORD PATTERNS

- **60%** follow "Word + Numbers + !" pattern¹
- Seasonal variations: "Summer2024!", "Winter2023!"
- Company name patterns: "AcmeCorp123!"
- **Actionable:** Add to custom forbidden list

SHARED PASSWORD PREVALENCE

- **15-25%** of passwords reused across accounts²
- Service accounts = biggest offenders
- Cross-domain sharing indicates systemic issues
- **Actionable:** Migrate to gMSA, enforce unique passwords

POLICY COMPLIANCE VIOLATIONS

- **40-50%** of passwords meet minimum requirements only³
- Length clustering at 8 characters (minimum)
- Non-expiring passwords: 10-20% of accounts
- **Actionable:** Increase minimum length to 14-15 chars

HIBP BREACH EXPOSURE

- **15-20%** of corporate passwords found in breaches⁴
- Tier 1-3 passwords = immediate risk
- Users reuse personal passwords at work
- **Actionable:** Deploy breach database checking

POLICY ACTIONS TO TAKE (PART 1)

1. INCREASE MINIMUM LENGTH

- **NIST:** 15 characters recommended¹
- **Microsoft:** 14 characters minimum²
- Length > complexity for security
- Passphrases easier to remember than complex passwords

2. BAN COMMON PASSWORDS

- **Microsoft Entra ID Password Protection**³
- Global banned list (~500 passwords, auto-updated)
- Custom banned list (up to 1,000 passwords)
- Add patterns found in YOUR audit

3. ENABLE BREACH DATABASE CHECKING

- **NIST requirement:** Check against breach databases¹
- Options: Entra ID Password Protection, Specops, Enzoic
- Prevent passwords from HIBP database
- Block on password change, not just creation

POLICY ACTIONS TO TAKE (PART 2)

4. ELIMINATE FORCED COMPLEXITY

- **NIST:** Composition rules are counter-productive¹
- Users create predictable patterns (Capital+word+123!)
- Focus on length + breach checking instead
- Allow any printable ASCII + Unicode

5. REMOVE PERIODIC EXPIRATION

- **NIST & Microsoft:** No forced rotation^{1,2}
- Forced changes = weaker passwords (Password123 → Password124)
- Change only on evidence of compromise
- Reduces user frustration, improves security

Use Audit Data to Justify Policy Changes: "15% of our passwords are in HIBP database" → "Deploy breach checking immediately"

Show management the numbers, reference NIST/Microsoft guidance, propose specific actions

MODERN POLICY RECOMMENDATIONS

1. LENGTH: 14-15 CHARACTERS

- **NIST:** 15 chars recommended, 64 max¹
- **Microsoft:** 14 chars minimum, 256 max²

2. COMPLEXITY

- Encourage users to think in passphrases, and long multiple words, numbers and special characters

3. PASSWORD EXPIRATION

- Forced rotation every 90 days = weaker passwords
- Change on compromise
- Change at least semi-annually

4. BREACH CHECKING

- **Microsoft Entra ID Password Protection**¹
- Global list (~500) + Custom list (1,000)

5. MFA: ALWAYS ENABLE

- **Microsoft Research:** Blocks 99.9% of attacks (caviots here though...)²
- Enforce for all privileged accounts minimum

PASSWORD! AT THE DISCO: SUMMARY

THE PROBLEM

Traditional password auditing leaves massive gaps between statistics and actionable intelligence

THE SOLUTION

Automated risk-based analysis

- Integration: One tool, all the data
- Automation: Minutes instead of hours
- Prioritization: Know what to fix first

THE RESULT

- **Hours of manual work** → Minutes of automated processing
- **5,000 weak passwords** → 12 critical accounts to fix first
- **Statistics** → Actionable intelligence

NEXT STEPS...

- **GitHub Repository:**

- github.com/passwordthedisco/PasswordAtTheDisco

- **Documentation:**

- Setup guide in README
- Full documentation in docs/

- **Requirements:**

- Python 3.10+
- BloodHound Community Edition
- HIBP database (42GB download)
- Hashcat (Coming soon!)

- **Contributing:**

- Pull Requests Welcome!
- Report issues, suggest features, contribute code

